

Report Doom Final Challenge

Student Number: K51843919

Student Name: Lukas Steu

Setup

I ran the notebook mainly in Google Colab with a Python 3 runtime and used a L4 GPU when available. The task used the custom ViZDoom multiplayer environment from the provided jku.wad repository. The local evaluation was done in the ROOM map with 4 easy bots and an episode timeout of 2000 steps, which is close to the challenge setup.

The final agent used RGB screen input together with the depth buffer. I also used 4 stacked frames to give the model temporal information. With RGB plus depth, one frame has 4 channels, so 4 stacked frames resulted in 16 input channels. Since the environment returned stacked frames as (C, T, H, W), I added a helper function that reshapes the input to (C*T, H, W) before passing it to the CNN.

Changes of the Code

First, I implemented a stronger DQN model instead of the small baseline network. The model uses several convolutional layers, adaptive average pooling and a small fully connected head. It outputs one Q-value for each of the 8 discrete actions. Since this is a Q-value model, the evaluation uses the action with the highest Q-value.

I added support for stacked observations throughout the code. The flattening function is used during model initialization, epsilon-greedy action selection, evaluation, replay buffer storage and ONNX export. This was necessary because the CNN expects a normal (B, C, H, W) input.

I also improved the DQN training procedure. I used a target network, a replay buffer, Huber loss and gradient clipping. The target update was changed to Double DQN: the online model selects the next action, while the target model evaluates it. To reduce memory usage, replay buffer observations are stored on the CPU as float16.

I tested Behavior Cloning as a warm start using the provided dataset. I implemented a BC dataset loader, frame stacking for BC samples and action-distribution diagnostics. However, the BC policy was not stable. In several runs it collapsed to mostly one action, such as only turning, only moving forward or mostly shooting. Because of that, Behavior Cloning was not used as the final main approach.

After that, I focused on DQN with reward shaping. The base reward still follows the task idea: positive reward for hits, a large reward for frags and a penalty for hits taken. I added small training-only shaping rewards and penalties to reduce bad behaviour, such as turning forever or shooting without hitting. The reward used for training was scaled before being stored in the replay buffer, while evaluation still used the original environment reward.

For model selection, I added server-like evaluation during training. If a model achieved a better quick evaluation score, it was evaluated again over more episodes before saving it as the best model. This was important because Doom episodes are noisy and a single lucky episode can give a misleading score.

Results

The Behavior Cloning experiments showed that validation accuracy alone was not a reliable metric. Some BC models had acceptable validation accuracy but performed badly in the environment because they collapsed to a small set of actions. The action-distribution diagnostic was useful to detect this problem.

The final DQN agent sometimes learned useful behaviour and was able to get hits or frags, but the performance was still unstable across different seeds. Some episodes achieved clearly positive rewards, while others were close to zero or negative. The final local evaluation result was:

Final local score over 10 episodes: -0.52

Best confirmed training evaluation score: -0.73

Runtime

The setup took about 1min 30s. The Behavior Cloning experiments took about 45 min, but were not used as the final strategy. The final DQN training took about 4h 35min for 210 episodes with a connected hosted time of type L4. The trained model was exported as model.onnx and submitted together with the code and this report.